

These structural constraints outline how to build a production-grade system rather than a fragile demo. The explicit focus on a `BUILDLOG.md`, an independent infrastructure blueprint, and strict compliance with zero-cost data parsing mirrors how enterprise applications manage stochastic AI behaviors.

To help you hit the ground running with **Phase 1**, here is your ready-to-use **One-Page Design Document**. You can copy this directly into your repository as a base architecture template.



Architecture Design Document: Content Matching Engine

1. Problem Statement

Automated CMS platforms frequently misallocate visual assets to articles because they rely on fragile, manual keyword string-matching or literal file naming conventions (e.g., matching a post containing the word "fox" to a picture of a "wolf" because the file was poorly labeled).

This system resolves the issue by extracting structured semantic profiles from image payloads using a multimodal model, executing mathematical vector comparisons against target articles, and implementing an explicit **Mismatch Guard** to prioritize safe rejection over an inaccurate match.

2. Core Constraints & Non-Goals

- **In-Scope Goals:** Enforce strict schema validation on all visual analyses; operate entirely within a \$0 infrastructure footprint; maintain a strict local execution engine for vector comparisons; isolate API tokens via environment variables.
- **Explicit Non-Goals:** This system is **not** an image search engine, an asset gallery explorer, a bulk tagging manager, or a real-time user-facing frontend interface. It is a deterministic, backend decision engine.

3. Data Model Schema

The internal system stores processing states in memory or a lightweight local cache structure using the following definitions:

```
json
{
  "asset_id": "string (UUID or file hash)",
  "file_path": "string",
  "visual_profile": {
```

```

    "exact_subject": "string",
    "literal_description": "string",
    "defining_traits": ["string"]
  },
  "processing_metadata": {
    "tokens_consumed": "integer",
    "estimated_cost_usd": "float",
    "timestamp": "string (ISO 8601)"
  }
}

```

Use code with caution.

4. API Surface Layout

The internal engine exposes two main functions to process batch workflows:

- `ingest_image_library(directory_path: str) -> List[dict]`
 - Loops over target directories using async chunking limits to prevent rate limit exceptions. Returns structural profile blocks.
- `evaluate_content_fit(article_text: str, profiles: List[dict]) -> dict`
 - Executes the core Challenge 2 Relevance Gate. Returns a definitive match or a structured rejection notification.

Your Repository Setup Tasks

To stay aligned with the five evaluative ground rules from day one, initialize your standalone GitHub repository with this structural footprint:

text

```

├─ .env                <- Stored locally. NEVER COMMIT THIS. Contains GEMINI_API_KEY.
├─ .gitignore          <- Configured to ignore .env, local caches, and __pycache__ fold
├─ DESIGN.md           <- The architecture document defined right above.
├─ BUILDLOG.md         <- Your chronological workspace diary tracking AI tool interacti

```

```
|— requirements.txt      <- Unified environment dependencies (google-genai, sentence-tran
|— engine/              <- Core application pipeline source package
```

Use code with caution.

 Initializing Your Workspace

BUILDLOG.md

Start your audit trail immediately by adding your first entry. This protects you during the final demo verification and confirms your ownership of the codebase:

```
markdown

# 🛠️ Project Build Log

### [2026-08-25] Phase 1 Initialization
- **What I Did:** Established the dedicated public repository scaffold. Completed the core
- **AI Tool Assistance:** Used an AI collaborator to structure the JSON schema requirement
- **Human Course Corrections:** Validated that the `google-genai` modern SDK implementatio
```

Use code with caution.

Are you planning to build out the background batch workflow next using Python's `asyncio`